



Instituto Politécnico Nacional
"La Técnica al Servicio de la Patria"



robótica y mecatrónica



INSTITUTO POLITÉCNICO NACIONAL CENTRO DE INVESTIGACIÓN EN COMPUTACIÓN LAB. ROBÓTICA Y MECATRÓNICA

PRACTICAS

DRON

Título específico de lo que ud. hizo

Autores:

Rodrigo Emmanuel Arellano Flores

Profesores:

Prof. Dr. Juan Humberto Sossa Azuela

CDMX.
22 de diciembre de 2025

Índice

1. Resumen.	3
2. Introducción.	3
3. Información preliminar.	3
3.1. Orden del repositorio	3
3.2. Componentes del cuadricóptero	3
3.3. Control del cuadricóptero	4
3.4. Procesamiento de sensores	5
3.4.1. IMU	5
3.4.2. Barómetro	5
4. Metodologías.	6
4.1. Medición de empuje	6
4.2. Detección y seguimiento	8
4.3. Dinámica	9
4.3.1. Linealización	10
4.4. Algoritmos de control	10
4.4.1. PD	10
4.4.2. PID	12
4.4.3. LQR y LQI	13
4.5. Resultados.	14
5. Conclusiones.	17
Apéndices	18
A. Memoria de cálculos.	18
B. Algoritmos.	18
C. Planos	25

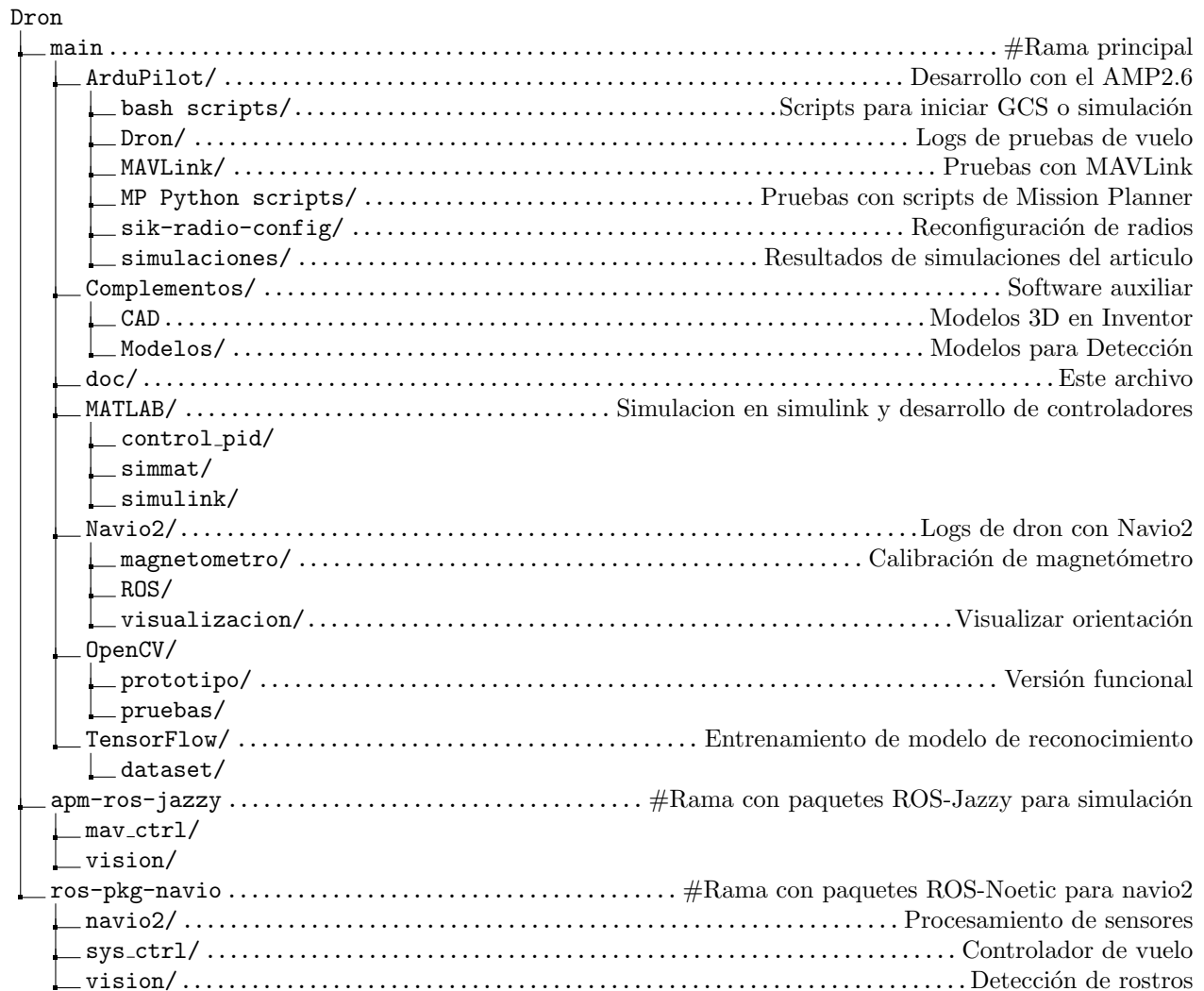
1. Resumen.

2. Introducción.

3. Información preliminar.

3.1. Orden del repositorio

El repositorio se encuentra en [GitHub](#) y se divide en tres ramas



El HAT Navio2 es compatible con las Raspberry Pi 2, 3 y 4. La imagen del sistema necesaria se encuentra [aquí](#). El sistema no tiene interfaz gráfica y funciona en arquitectura armv7l.

Para poder establecer un acceso remoto al Raspberry del dron incluso fuera de red local, se puede usar el servicio VPN de [Tailscale](#).

3.2. Componentes del cuadricóptero

El dron cuenta con los siguientes componentes:

- 4 Motores sin escobillas (BLDC)
- 4 Control de velocidad electrónico (ESC)
- 1 Batería LiPo 11.1 V

- 1 Computadora de vuelo, cuenta con:
 - IMU (acelerómetro, giroscopio y magnetómetro)
 - Barómetro
- 1 Chasis
- 2 Helices CW
- 2 Helices CCW

3.3. Control del cuadricóptero

El dron utiliza un marco X y el orden de los motores así como su dirección de giro se puede ver en la figura 1.

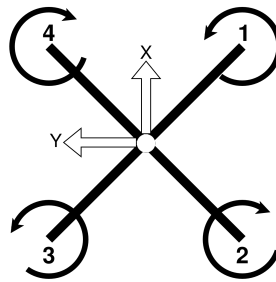


Figura 1: Marco

El par y empuje generados por cada motor están dados por:

$$F_i = K_F \omega_i^2 \quad (1)$$

$$\tau_i = K_M \omega_i^2 \quad (2)$$

De modo que la relación entre la acción de cada motor y las entradas de control se relacionan de la siguiente forma

$$\begin{bmatrix} F \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} K_F & K_F & K_F & K_F \\ -K_F L & -K_F L & K_F L & K_F L \\ -K_F L & K_F L & K_F L & -K_F L \\ -K_M & K_M & -K_M & K_M \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} \quad (3)$$

El modelo dinámico del dron se puede linealizar el modelo del dron mediante la serie de Taylor [1]

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})_{\mathbf{x}_0, \mathbf{u}_0} + \left. \frac{\partial}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right|_{(\mathbf{x}_0, \mathbf{u}_0)} (\mathbf{x} - \mathbf{x}_0) + \left. \frac{\partial}{\partial \mathbf{u}} f(\mathbf{x}, \mathbf{u}) \right|_{(\mathbf{x}_0, \mathbf{u}_0)} (\mathbf{u} - \mathbf{u}_0) = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \quad (4)$$

$$\mathbf{x}_0 = \mathbf{0}_{12 \times 1}$$

$$\mathbf{u}_0 = [mg \ 0 \ 0 \ 0]^T$$

El modelo en espacio de estados se puede convertir a funciones de transferencia mediante

$$\mathbf{G}(s) = \mathbf{C} [s\mathbf{I} - \mathbf{A}]^{-1} \mathbf{B} + \mathbf{D} \quad (5)$$

El **controlador PD** tiene la siguiente estructura

$$PD(s) = K_P + K_D s = K_D(s + a) \quad (6)$$

$$K_P = aK_D$$

El **controlador PID** tiene la siguiente estructura

$$PID(s) = K_P + \frac{K_I}{s} + K_D s = K_D \frac{(s+a)(s+b)}{s} \quad (7)$$

$$K_I = abK_D$$

$$K_P = (a+b)K_D$$

Seguimiento de referencia en control en espacio de estados

Precompensador discreto [2]

$$\mathbf{u}(k) = -\mathbf{K}\mathbf{x}(k) + \mathbf{N}\mathbf{r}(k) \quad (8)$$

$$\mathbf{N} = [\mathbf{C}_d[\mathbf{I} - \mathbf{A}_d + \mathbf{B}_d\mathbf{K}]^{-1}\mathbf{B}_d]^{-1} \quad (9)$$

Modelo Interno para entrada escalón [2]

$$\begin{bmatrix} \mathbf{x}(k+1) \\ \mathbf{e}(k+1) \end{bmatrix} = \begin{bmatrix} \mathbf{A} & \mathbf{0} \\ -\mathbf{C} & \mathbf{I} \end{bmatrix} \begin{bmatrix} \mathbf{x}(k) \\ \mathbf{e}(k) \end{bmatrix} + \begin{bmatrix} \mathbf{B} \\ \mathbf{0} \end{bmatrix} \mathbf{u}(k) \quad (10)$$

$$\mathbf{u}(t) = -\mathbf{K}\mathbf{x}(k) - \mathbf{K}_e\mathbf{e}(k)$$

3.4. Procesamiento de sensores

3.4.1. IMU

Se pueden estimar los ángulos de orientación roll (ϕ) y pitch (θ) mediante un filtro complementario (11) que es una forma simple de fusionar la información del giroscopio y acelerómetro. Sin embargo, al ser una técnica simple esta limitado a movimientos suaves aun así puede ser utilizado para lograr estabilización del dron.

$$\theta = \alpha(\theta(k-1) + \omega_{gyro}T) + (1-\alpha)\theta_{acc}(k) \quad (11)$$

$$\phi_{acc} = \frac{acc_y}{\sqrt{acc_x^2 + acc_z^2}} \quad (12)$$

$$\theta_{acc} = -\frac{acc_x}{\sqrt{acc_y^2 + acc_z^2}} \quad (13)$$

El rumbo (yaw $\rightarrow \psi$) se puede obtener mediante

$$\psi = \text{atan2}(mag_y, mag_x) - \delta \quad (14)$$

La declinación magnética (δ) se puede obtener de [3]. Una mejor forma de obtener la orientación espacial del dron es mediante un algoritmo AHRS (Attitude and Heading Reference System), el filtro Mahony y Madgwick son dos excelentes opciones por su bajo costo computacional, se selecciona el filtro Madgwick ya que es más preciso que el Mahony y Kalman [4], aunque no mejor que el Kalman extendido [5]. El algoritmo proporciona la orientación en cuaterniones, los cuales pueden ser convertidos a ángulos de Euler mediante.

$$\phi = \text{atan2}(2(q_0q_1 + q_2q_3), 1 - 2(q_1q_1 + q_2q_2)) \quad (15)$$

$$\theta = \text{asin}(2(q_0q_2 - q_3q_1)) \quad (16)$$

$$\psi = \text{atan2}(2(q_0q_3 + q_1q_2), 1 - 2(q_2q_2 + q_3q_3)) \quad (17)$$

3.4.2. Barómetro

La altura ortométrica [6] se calcula con la información del barómetro mediante la siguiente ecuación

$$H_b = \frac{T_s}{k_T} \left(\frac{p_b \frac{Rk_T}{g_0}}{p_s} - 1 \right) \quad (18)$$

Donde:

$$R \ 287.1 \text{ J/kgK}$$

$$k_T \ 6.5 \times 10^{-3} \text{ K/m}$$

$$g_0 \ 9.80665 \text{ m/s}^2$$

$$p_s \ 101.325 \text{ kPa}$$

$$T_s \ 288.15 \text{ K}$$

4. Metodologías.

4.1. Medición de empuje

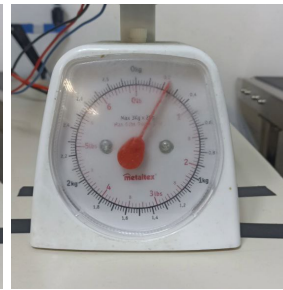
Para estimar K_F de (1) se toman medidas del empuje generado cada 10 % de cambio en la velocidad del motor. Las siguientes medidas fueron realizadas con a 11.2 V, el motor y hélice tienen una masa de 120 g.



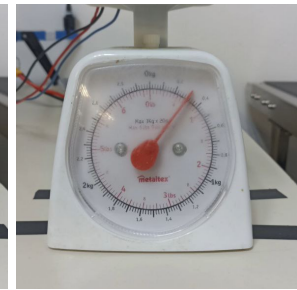
(a) 1100 ms $\rightarrow$ 140 g



(b) 1200 ms $\rightarrow$ 170 g



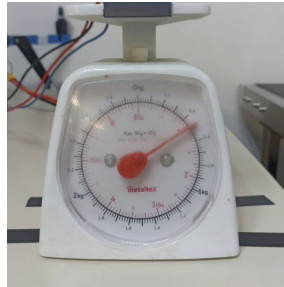
(c) 1300 ms $\rightarrow$ 220 g



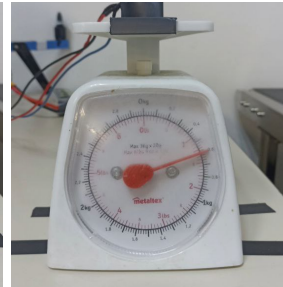
(d) 1400 ms $\rightarrow$ 300 g



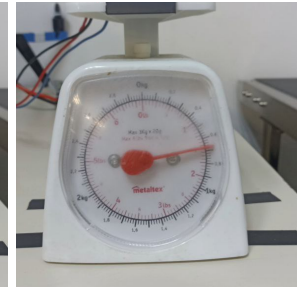
(e) 1500 ms $\rightarrow$ 400 g



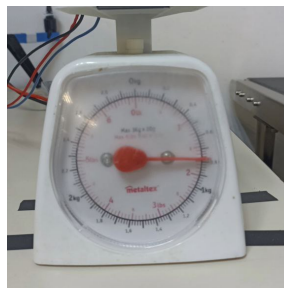
(f) 1600 ms $\rightarrow$ 490 g



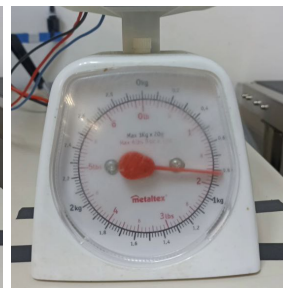
(g) 1700 ms $\rightarrow$ 600 g



(h) 1800 ms $\rightarrow$ 670 g



(i) 1900 ms $\rightarrow$ 770 g



(j) 2000 ms $\rightarrow$ 820 g

Figura 2: Medición de empuje

Cuadro 1: Medición prueba de empuje

Señal (%)	Medido (g)	Empuje (g)
0	120	0
10	140	20
20	170	50
30	220	100
40	300	180
50	400	280
60	490	370
70	600	480
80	670	550
90	770	650
100	820	700

Con estos datos se puede ajustar la curva para obtener $K_F = 7,965 \text{ N}/\%^2$

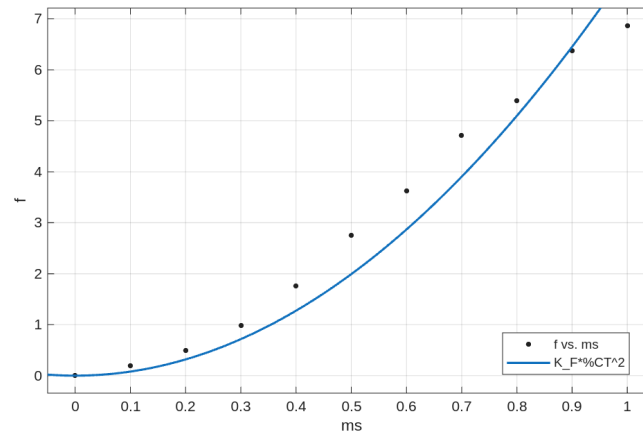
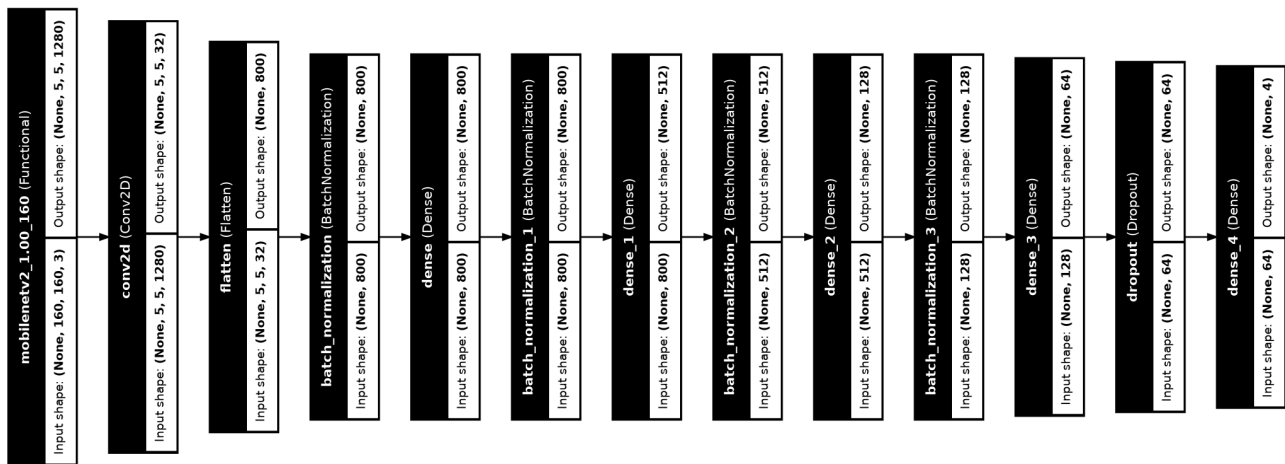
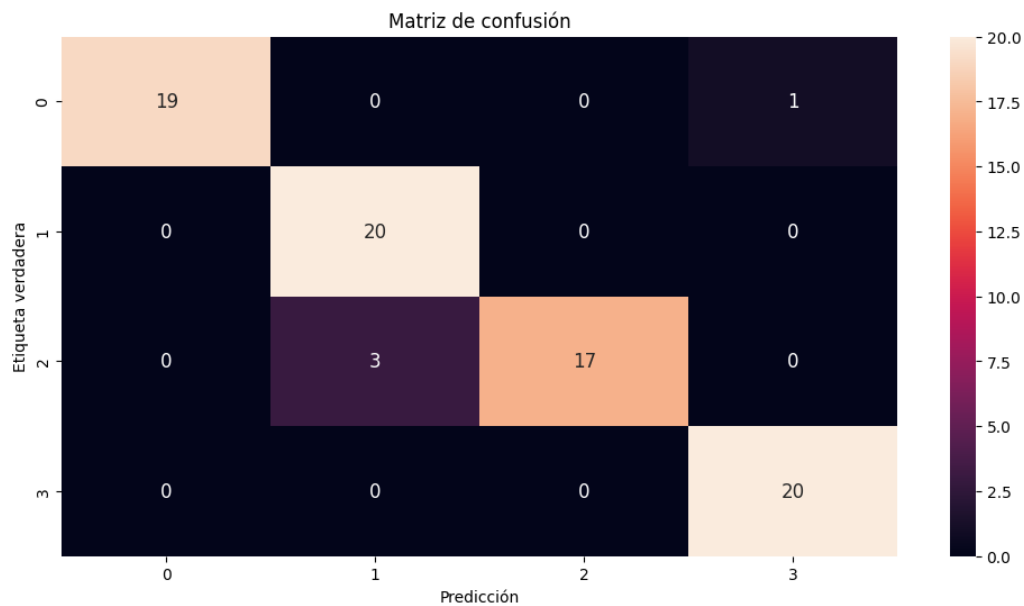


Figura 3: Ajuste de curva

4.2. Detección y seguimiento



(a) Arquitectura



(b) Matriz de confusión

Figura 4: Red entrenada v2.0

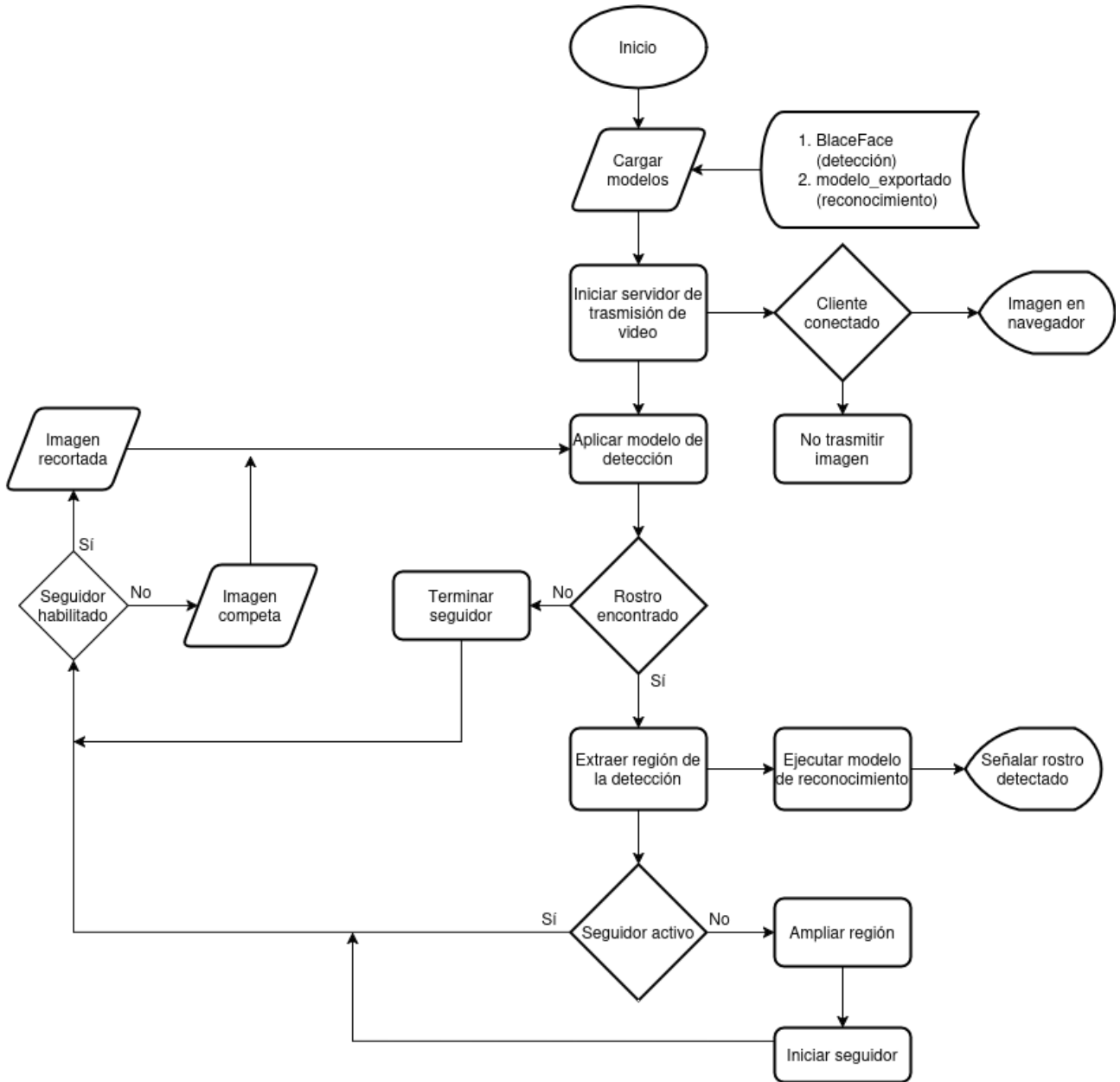


Figura 5: Programa

4.3. Dinámica

El modelo dinámico del cuadricóptero es

$$m\ddot{\mathbf{x}} = \mathbf{R}_{\phi,\theta,\psi} \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} - \mathbf{F}_f \quad (19)$$

$$M(\boldsymbol{\eta})\ddot{\boldsymbol{\eta}} = \boldsymbol{\tau} - C(\boldsymbol{\eta}, \dot{\boldsymbol{\eta}})\dot{\boldsymbol{\eta}} - \boldsymbol{\tau}_f \quad (20)$$

De (3) se obtiene la relación entre entradas de control y actuadores

$$\begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = \begin{bmatrix} \frac{1}{4K_F} & -\frac{1}{4K_F L} & -\frac{1}{4K_F L} & -\frac{1}{4K_M} \\ \frac{1}{4K_F} & -\frac{1}{4K_F L} & \frac{1}{4K_F L} & \frac{1}{4K_M} \\ \frac{1}{4K_F} & \frac{1}{4K_F L} & \frac{1}{4K_F L} & -\frac{1}{4K_M} \\ \frac{1}{4K_F} & \frac{1}{4K_F L} & -\frac{1}{4K_F L} & \frac{1}{4K_M} \end{bmatrix} \begin{bmatrix} F \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} \quad (21)$$

Cuadro 2: Parámetros del dron

m	1.6	kg
I_{xx}	0.021915027	kgm ²
I_{yy}	0.022124548	kgm ²
I_{zz}	0.042349865	kgm ²
L	0.23	m
K_F	7.965	N/% ²
K_M	0.5	Nm/% ²
K_m	50.0565	s ⁻¹

4.3.1. Linealización

Mediante la serie de Taylor (4) el modelo del cuadricóptero se puede expresar de forma lineal de la siguiente forma

$$\begin{bmatrix} \dot{\mathbf{r}} \\ \ddot{\mathbf{r}} \\ \dot{\boldsymbol{\eta}} \\ \ddot{\boldsymbol{\eta}} \end{bmatrix} = \begin{bmatrix} 0_{3 \times 3} & I_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} \\ 0_{3 \times 3} & 0_{3 \times 3} & G(\psi_d) & 0_{3 \times 3} \\ 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} & I_{3 \times 3} \\ 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} \end{bmatrix} \begin{bmatrix} \mathbf{r} \\ \dot{\mathbf{r}} \\ \boldsymbol{\eta} \\ \dot{\boldsymbol{\eta}} \end{bmatrix} + \begin{bmatrix} 0_{5 \times 1} & 0_{5 \times 3} \\ m^{-1} & 0_{1 \times 3} \\ 0_{3 \times 1} & 0_{3 \times 3} \\ 0_{3 \times 1} & I_d \end{bmatrix} \begin{bmatrix} F \\ \boldsymbol{\tau} \end{bmatrix} \quad (22)$$

donde:

$$\mathbf{r} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \quad \boldsymbol{\eta} = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}, \quad \boldsymbol{\tau} = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$$

$$G(\psi_d) = \begin{bmatrix} 0 & g & 0 \\ -g & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad I_d = \text{diag} \left(I_{xx}^{-1}, I_{yy}^{-1}, I_{zz}^{-1} \right)$$

4.4. Algoritmos de control

4.4.1. PD

El controlador se ajusta mediante las funciones de transferencia, que se obtienen mediante (5), con

$$\mathbf{C} = \begin{bmatrix} I_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} & 0_{3 \times 3} \\ 0_{3 \times 3} & 0_{3 \times 3} & I_{3 \times 3} & 0_{3 \times 3} \end{bmatrix}$$

$$\mathbf{D} = 0_{6 \times 4}$$

$$G_x(s) = \frac{g}{I_{yy}s^4} \quad (23)$$

$$G_y(s) = -\frac{g}{I_{xx}s^4} \quad (24)$$

$$G_z(s) = \frac{1}{ms^2} \quad (25)$$

$$G_\phi(s) = \frac{1}{I_{xx}s^2} \quad (26)$$

$$G_\theta(s) = \frac{1}{I_{yy}s^2} \quad (27)$$

$$G_\psi(s) = \frac{1}{I_{zz}s^2} \quad (28)$$

Dado que no se puede medir los desplazamientos en x y y , solo se realizan 4 de los 6 controladores (control de orientación y altura). Dado que el cambio de velocidad en los motores no es instantáneo, las ecuaciones (28)-(28) se multiplican por (29), que es la función de transferencia de los motores, para incluir el retraso.

$$\dot{\omega}(t) = K_m(\omega_d(t) - \omega(t)) \rightarrow \frac{\omega(s)}{\omega_d(s)} = \frac{K_m}{s + K_m} \quad (29)$$

Para ajustar los controladores se utiliza el método de colocación de las raíces, de modo que (25) a (28) se convierten en

$$K_{Dz} \frac{(s + a_z) \frac{K_m}{m}}{s^2(s + K_m)} \quad (30)$$

$$K_{D\phi} \frac{(s + a_\phi) \frac{K_m}{I_{xx}}}{s^2(s + K_m)} \quad (31)$$

$$K_{D\theta} \frac{(s + a_\theta) \frac{K_m}{I_{yy}}}{s^2(s + K_m)} \quad (32)$$

$$K_{D\psi} \frac{(s + a_\psi) \frac{K_m}{I_{zz}}}{s^2(s + K_m)} \quad (33)$$

Con la ayuda MATLAB se ajustan los ceros y ganancias hasta obtener los siguiente resultados.

Cuadro 3: Ganancias PD

	K_P	K_D
Altura	32.9186	20.994
Roll	1.498320	0.352480
Pitch	1.512645	0.355850
Yaw	2.895440	0.681153

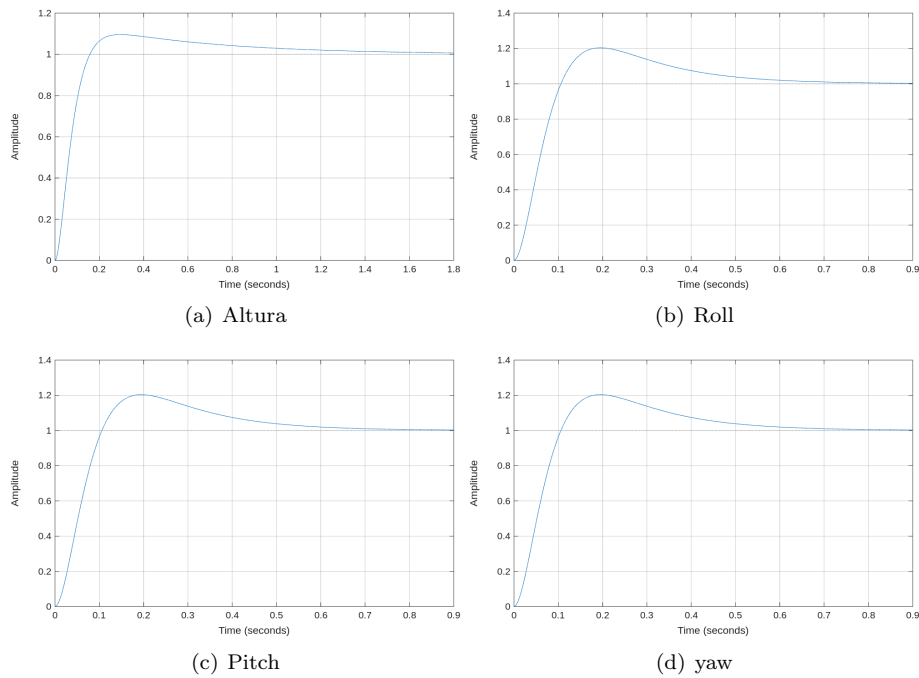


Figura 6: Respuesta escalón PD

4.4.2. PID

De forma similar se ajustan las ganancias del controlador, para este caso se usan

$$K_{Dz} \frac{(s + a_z)(s + b_z) \frac{K_m}{m}}{s^2(s + K_m)} \quad (34)$$

$$K_{D\phi} \frac{(s + a_\phi)(s + b_\phi) \frac{K_m}{I_{xx}}}{s^2(s + K_m)} \quad (35)$$

$$K_{D\theta} \frac{(s + a_\theta)(s + b_\theta) \frac{K_m}{I_{yy}}}{s^2(s + K_m)} \quad (36)$$

$$K_{D\psi} \frac{(s + a_\psi)(s + b_\psi) \frac{K_m}{I_{zz}}}{s^2(s + K_m)} \quad (37)$$

Cuadro 4: Ganancias PID

	K_P	K_I	K_D
Altura	29.28	0.8	38.21
Roll	1.016	0.08	0.2
Pitch	2.873	1.173	0.6391
Yaw	3.329	1.211	2.288

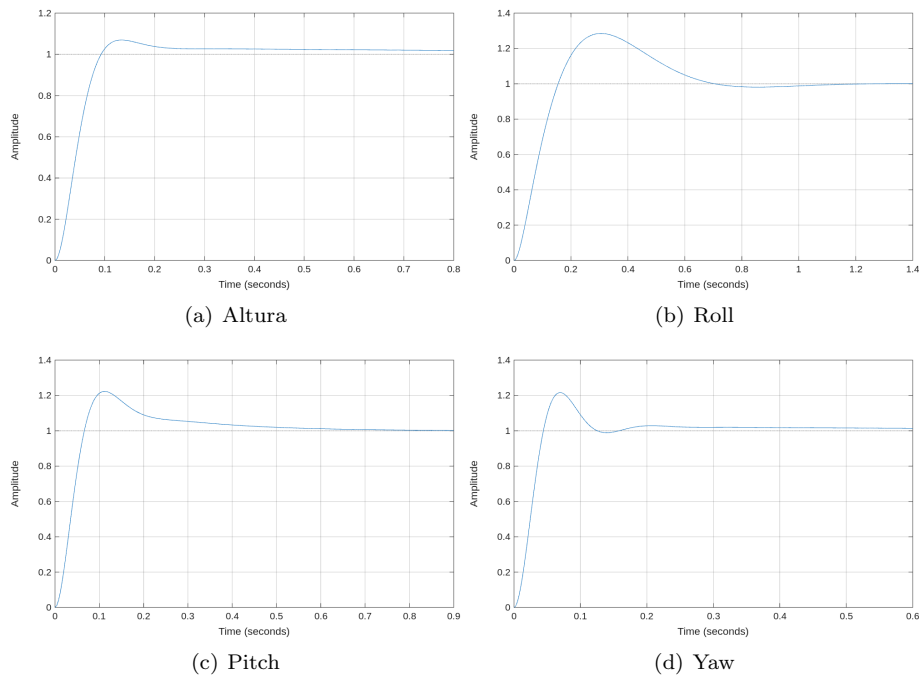


Figura 7: Respuesta escalón PID

4.4.3. LQR y LQI

Para controlar actitud (orientación) se diseña un controlador LQR con precompensador para seguimiento de referencias, el modelo en espacio de estados en tiempo continuo es

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \\ \ddot{\phi} \\ \ddot{\theta} \\ \ddot{\psi} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \phi \\ \theta \\ \psi \\ \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ \frac{1}{I_{xx}} & 0 & 0 \\ 0 & \frac{1}{I_{yy}} & 0 \\ 0 & 0 & \frac{1}{I_{zz}} \end{bmatrix} \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$$

Se convierte a discreto utilizando zero order hold (ZOH) y tiempo de muestreo de $1/510 = 0,0020$ s, el control de retroalimentación de estados se diseña con LQR y con

$$\mathbf{Q} = \text{diag}([0,0175^{-2} \quad 0,0175^{-2} \quad 0,0175^{-2} \quad 0,2618^{-2} \quad 0,2618^{-2} \quad 0,2618^{-2}])$$

$$\mathbf{R} = \text{diag}([0,0625 \quad 0,0625 \quad 0,0625])$$

Las ganancias del controlador son:

$$\mathbf{K} = \begin{bmatrix} 119,2106 & 0 & 0 & 8,2696 & 0 & 0 \\ 0 & 119,8505 & 0 & 0 & 8,3153 & 0 \\ 0 & 0 & 159,7183 & 0 & 0 & 11,2652 \end{bmatrix}$$

$$\mathbf{N} = \begin{bmatrix} 119,2106 & 0 & 0 \\ 0 & 119,8505 & 0 \\ 0 & 0 & 159,7183 \end{bmatrix}$$

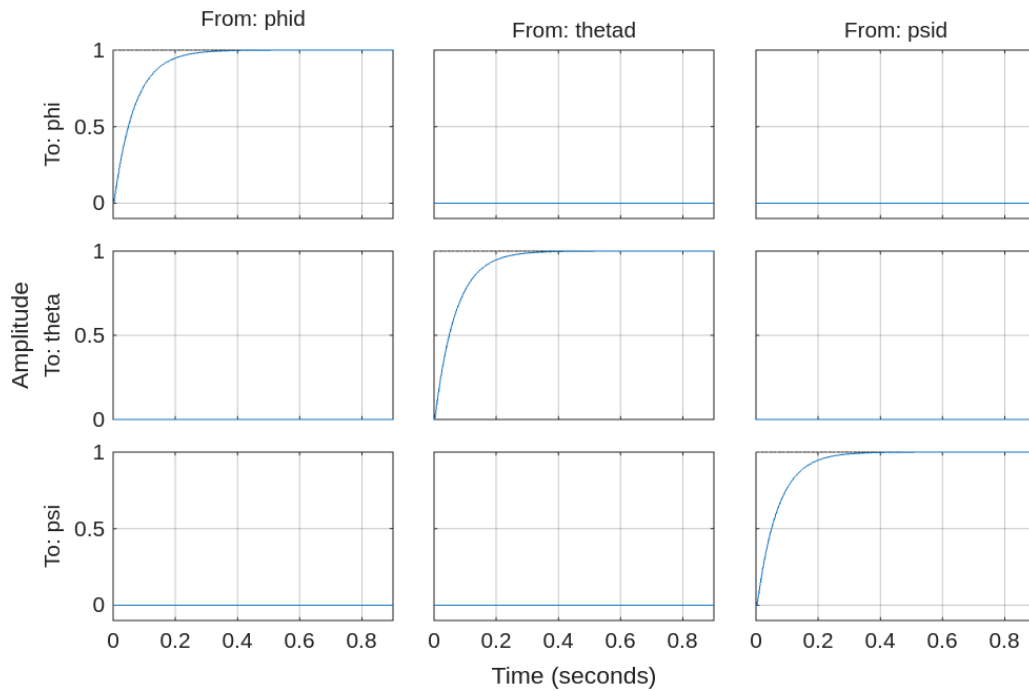


Figura 8: Respuesta escalón LQR para control de actitud

Para el control de altura se diseña un controlador LQI, el modelo en tiempo continuo es

$$\begin{bmatrix} \dot{z} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} z \\ \dot{z} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} F$$

$$\mathbf{Q} = \text{diag}([1 \quad 0,0044 \quad 25])$$

$$\mathbf{R} = 0,2500$$

Las ganancias del controlador son:

$$\mathbf{K} = [224,0753 \quad 26,2236 \quad -9,1805]$$

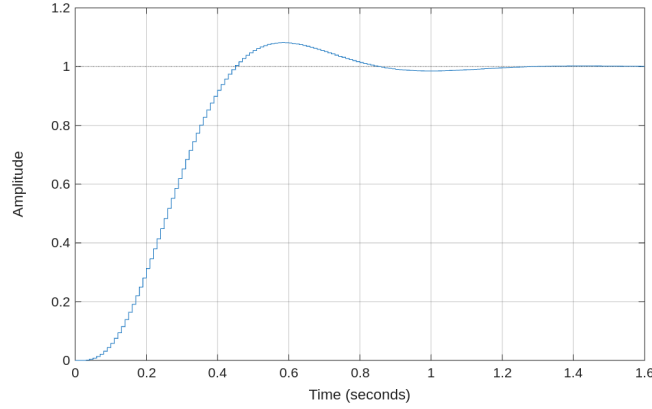


Figura 9: Respuesta escalón LQI para control de altura

4.5. Resultados.

Controlador PD:

$$u_z(k) = K_{Pz}e(k) + K_{Dz}\frac{e(k) - e(k-1)}{T} + mg \quad (38)$$

$$u_\eta(k) = K_{P\eta}e(k) + K_{D\eta}\frac{e(k) - e(k-1)}{T} \quad (39)$$

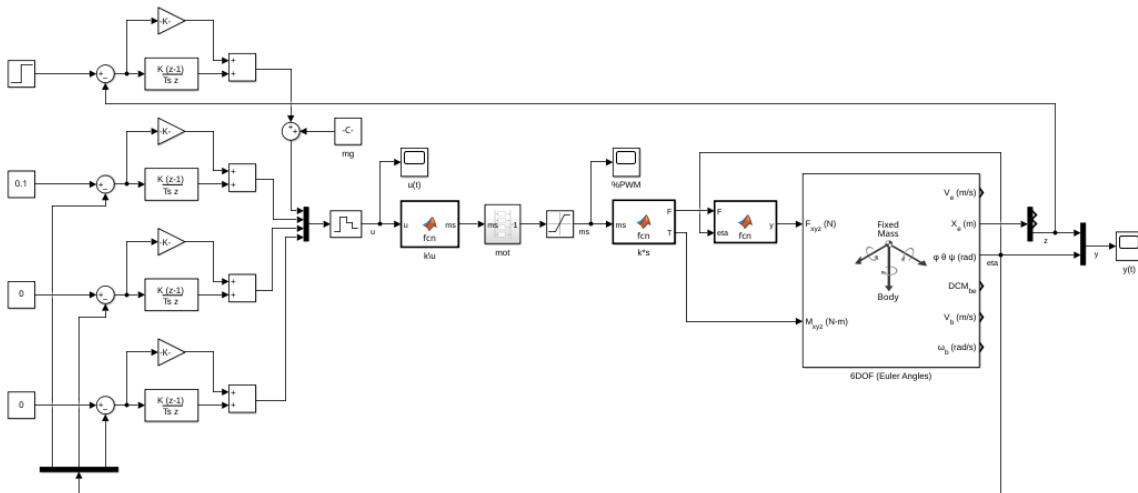


Figura 10: PD en Simulink. Referencias $z = 1 \text{ m}$, $\eta = [0,1 \quad 0 \quad 0]^T \text{ rad}$

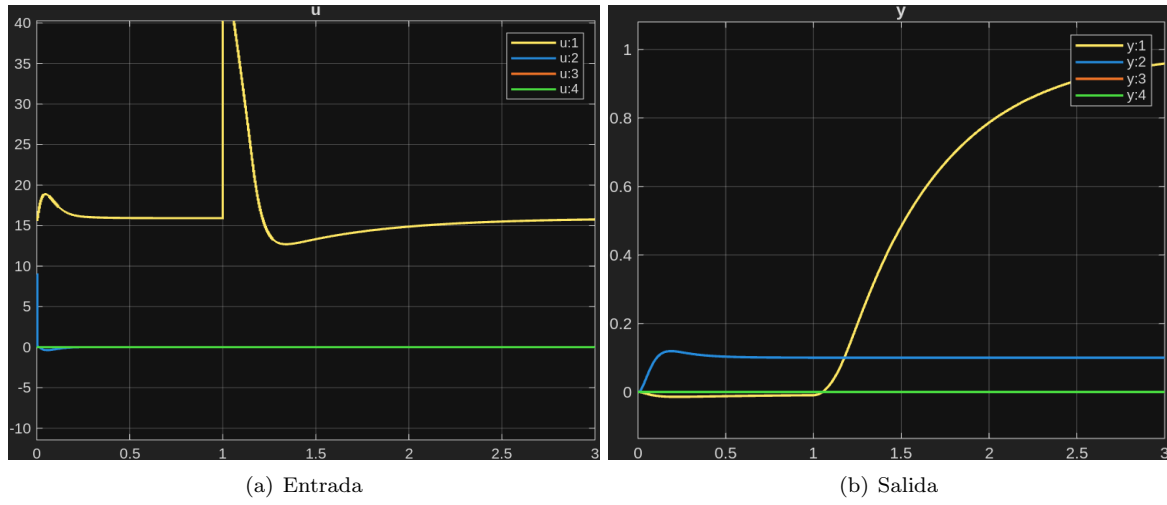


Figura 11: Simulación PD, controlador a 510 Hz

Controlador PID:

$$u_z(k) = K_{Pz}e_z(k) + K_{Iz}[e_z(k-1) + Te_z(k)] + K_{Dz}\frac{e_z(k) - e_z(k-1)}{T} + mg \quad (40)$$

$$u_\eta(k) = K_{P\eta}e_\eta(k) + K_{I\eta}[e_\eta(k-1) + Te_\eta(k)] + K_{D\eta}\frac{e_\eta(k) - e_\eta(k-1)}{T} \quad (41)$$

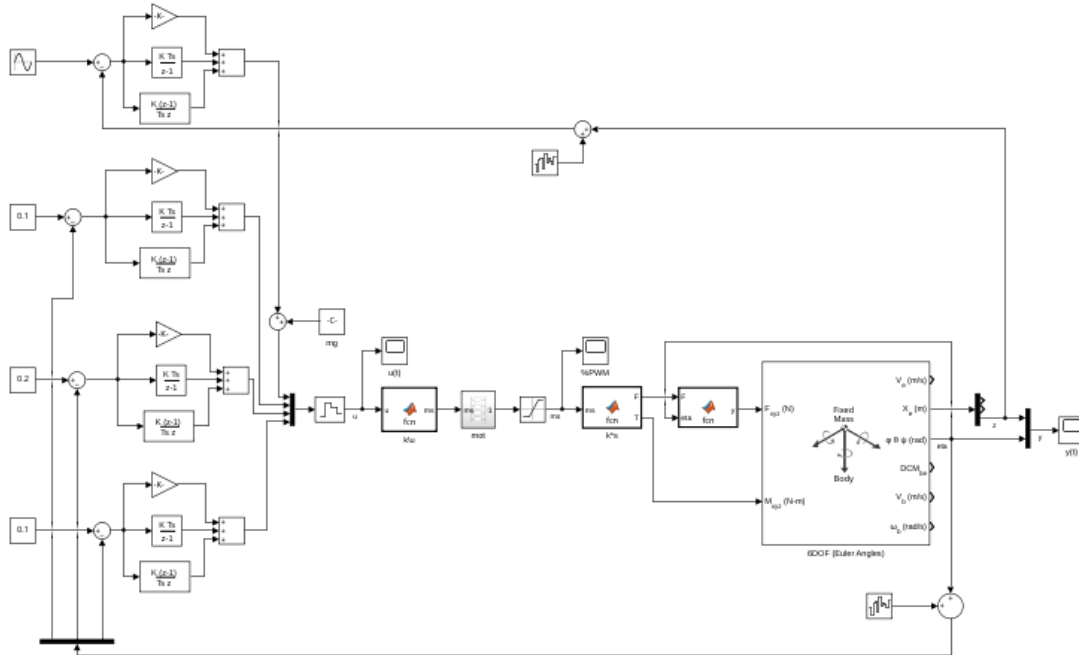


Figura 12: PID en Simulink (la información de los sensores tiene ruido). Referencias $z = 1 + 0,5\sin(2t)$ m, $\eta = [0,1 \ 0,2 \ 0,1]$

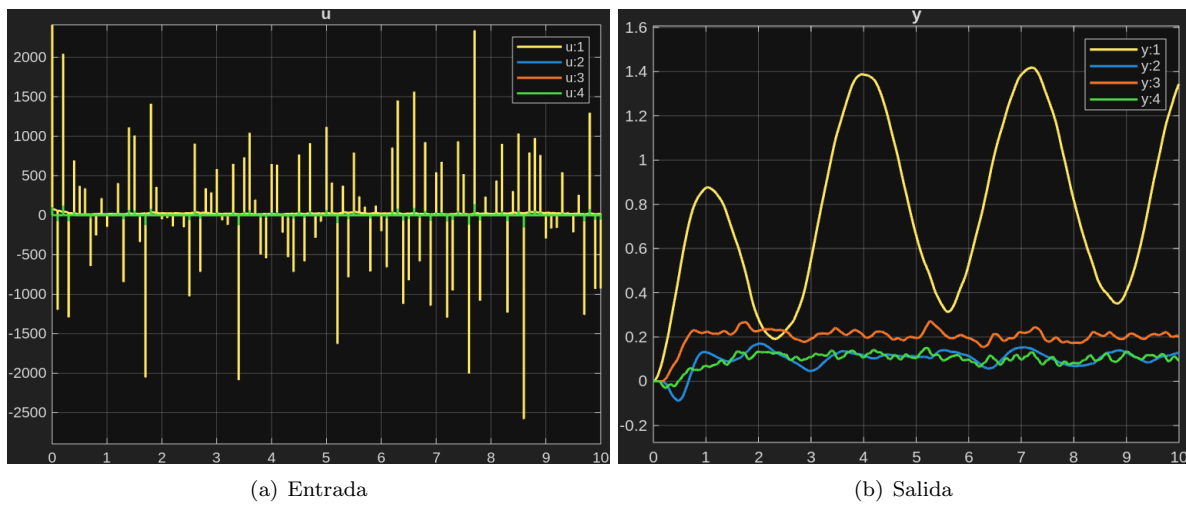


Figura 13: Simulación PID, controlador a 510 Hz

Controlador LQI

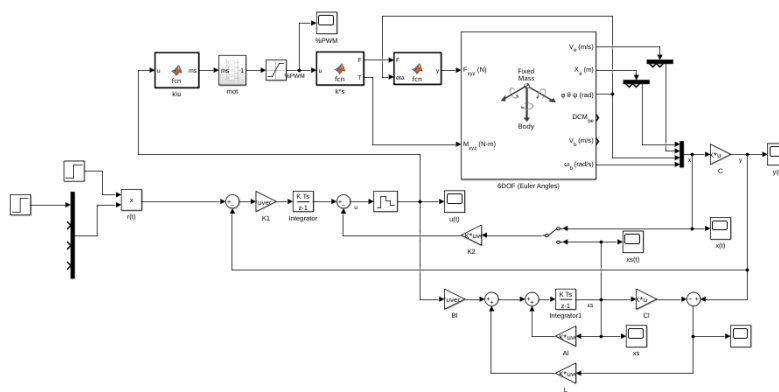


Figura 14: LQI en Simulink

5. Conclusiones.

**When esto es lo que pasa
cuando nunca te rindes:**



Figura 15: Esto es lo que pasa cuando nunca te rindes

Referencias

- [1] C. Powers, D. Mellinger y V. Kumar, «Quadrotor Kinematics and Dynamics,» en *Handbook of Unmanned Aerial Vehicles*, Springer, Dordrecht, 2015, págs. 307-328, ISBN: 978-90-481-9707-1. DOI: [10.1007/978-90-481-9707-1_71](https://doi.org/10.1007/978-90-481-9707-1_71). visitado 7 de dic. de 2025. dirección: https://link.springer.com/rwe/10.1007/978-90-481-9707-1_71.
- [2] M. S. Fadali y A. Visioli, «Servo problem,» en *Digital control engineering Analysis and design*, 2.^a ed., Elsevier, 2013, págs. 367-371, ISBN: 978-0-12-394391-0.
- [3] N. G. D. Center. «NCEI Geomagnetic Calculators,» visitado 17 de dic. de 2025. dirección: <https://www.ngdc.noaa.gov/geomag/calculators/magcalc.shtml#declination>.
- [4] S. O. H. Madgwick, A. J. L. Harrison y R. Vaidyanathan, «Estimation of IMU and MARG Orientation Using a Gradient Descent Algorithm,» en *2011 IEEE International Conference on Rehabilitation Robotics*, jun. de 2011, págs. 1-7. DOI: [10.1109/ICORR.2011.5975346](https://doi.org/10.1109/ICORR.2011.5975346). visitado 7 de dic. de 2025. dirección: <https://ieeexplore.ieee.org/document/5975346>.
- [5] A. Cavallo et al., «Experimental Comparison of Sensor Fusion Algorithms for Attitude Estimation,» *IFAC Proceedings Volumes*, 19th IFAC World Congress, vol. 47, n.º 3, págs. 7585-7591, 1 de ene. de 2014, ISSN: 1474-6670. DOI: [10.3182/20140824-6-ZA-1003.01173](https://doi.org/10.3182/20140824-6-ZA-1003.01173). visitado 8 de dic. de 2025. dirección: <https://www.sciencedirect.com/science/article/pii/S1474667016428089>.
- [6] P. D. Groves, «Barometric Altimeter,» en *Principles of GNSS, inertial, and multisensor integrated systems*, 2.^a ed., Artech House, 2013, págs. 230-231, ISBN: 978-1-60807-005-3.

Apéndices

A. Memoria de cálculos.

B. Algoritmos.

Listing 1: AHRS Madwgick.h

```

1  //
   =====

2  // MadgwickAHRS.h
3  //
   =====

4  //
5  // Implementation of Madgwick's IMU and AHRS algorithms.
6  // See: http://www.x-io.co.uk/node/8#open\_source\_ahrs\_and\_imu\_algorithms
7  //
8  // Date          Author          Notes
9  // 29/09/2011    SOH Madgwick    Initial release
10 // 02/10/2011    SOH Madgwick    Optimised for reduced CPU load
11 //
12 //
   =====

13 #ifndef MadgwickAHRS_h
14 #define MadgwickAHRS_h
15
16 //
   -----

17 // Variable declaration
18
19 extern volatile float beta;           // algorithm gain
20 extern volatile float q0, q1, q2, q3; // quaternion of sensor frame
   relative to auxiliary frame
21
22 //
   -----

23 // Function declarations
24
25 void MadgwickAHRSupdate(float gx, float gy, float gz, float ax, float ay,
   float az, float mx, float my, float mz);
26 void MadgwickAHRSupdateIMU(float gx, float gy, float gz, float ax, float ay,
   float az);
27
28 #endif
29 //
   =====

30 // End of file
31 //

```

Listing 2: AHRS Madgwick.c

```

1  //
2  // =====
3  //
4  //
5  // Implementation of Madgwick's IMU and AHRS algorithms.
6  // See: http://www.x-io.co.uk/node/8#open\_source\_ahrs\_and\_imu\_algorithms
7  //
8  // Date          Author          Notes
9  // 29/09/2011    SOH Madgwick     Initial release
10 // 02/10/2011    SOH Madgwick     Optimised for reduced CPU load
11 // 19/02/2012    SOH Madgwick     Magnetometer measurement is normalised
12 //
13 // =====
14
15 //
16 // -----
17
18 // Header files
19 #include "AHRS/MadgwickAHRS.h"
20 #include <math.h>
21 //
22 // -----
23
24 // Definitions
25 #define sampleFreq  512.0f          // sample frequency in Hz
26 #define betaDef      0.1f           // 2 * proportional gain
27 //
28 // -----
29
30 // Variable definitions
31 volatile float beta = betaDef;      // 2 *
32                                     // proportional gain (Kp)
33 volatile float q0 = 1.0f, q1 = 0.0f, q2 = 0.0f, q3 = 0.0f; // quaternion of
34                                     // sensor frame relative to auxiliary frame
35 //
36 // -----
37
38 // Function declarations
39

```

```

36 float invSqrt(float x);
37
38 //
=====

39 // Functions
40
41 //
-----

42 // AHRS algorithm update
43
44 void MadgwickAHRSupdate(float gx, float gy, float gz, float ax, float ay,
    float az, float mx, float my, float mz) {
45     float recipNorm;
46     float s0, s1, s2, s3;
47     float qDot1, qDot2, qDot3, qDot4;
48     float hx, hy;
49     float _2q0mx, _2q0my, _2q0mz, _2q1mx, _2bx, _2bz, _4bx, _4bz, _2q0, _2q1
        , _2q2, _2q3, _2q0q2, _2q2q3, q0q0, q0q1, q0q2, q0q3, q1q1, q1q2,
        q1q3, q2q2, q2q3, q3q3;
50
51     // Use IMU algorithm if magnetometer measurement invalid (avoids NaN in
        magnetometer normalisation)
52     if((mx == 0.0f) && (my == 0.0f) && (mz == 0.0f)) {
53         MadgwickAHRSupdateIMU(gx, gy, gz, ax, ay, az);
54         return;
55     }
56
57     // Rate of change of quaternion from gyroscope
58     qDot1 = 0.5f * (-q1 * gx - q2 * gy - q3 * gz);
59     qDot2 = 0.5f * (q0 * gx + q2 * gz - q3 * gy);
60     qDot3 = 0.5f * (q0 * gy - q1 * gz + q3 * gx);
61     qDot4 = 0.5f * (q0 * gz + q1 * gy - q2 * gx);
62
63     // Compute feedback only if accelerometer measurement valid (avoids NaN
        in accelerometer normalisation)
64     if(!((ax == 0.0f) && (ay == 0.0f) && (az == 0.0f))) {
65
66         // Normalise accelerometer measurement
67         recipNorm = invSqrt(ax * ax + ay * ay + az * az);
68         ax *= recipNorm;
69         ay *= recipNorm;
70         az *= recipNorm;
71
72         // Normalise magnetometer measurement
73         recipNorm = invSqrt(mx * mx + my * my + mz * mz);
74         mx *= recipNorm;
75         my *= recipNorm;
76         mz *= recipNorm;
77
78         // Auxiliary variables to avoid repeated arithmetic
79         _2q0mx = 2.0f * q0 * mx;
80         _2q0my = 2.0f * q0 * my;
81         _2q0mz = 2.0f * q0 * mz;

```

```

82     _2q1mx = 2.0f * q1 * mx;
83     _2q0 = 2.0f * q0;
84     _2q1 = 2.0f * q1;
85     _2q2 = 2.0f * q2;
86     _2q3 = 2.0f * q3;
87     _2q0q2 = 2.0f * q0 * q2;
88     _2q2q3 = 2.0f * q2 * q3;
89     q0q0 = q0 * q0;
90     q0q1 = q0 * q1;
91     q0q2 = q0 * q2;
92     q0q3 = q0 * q3;
93     q1q1 = q1 * q1;
94     q1q2 = q1 * q2;
95     q1q3 = q1 * q3;
96     q2q2 = q2 * q2;
97     q2q3 = q2 * q3;
98     q3q3 = q3 * q3;
99
100    // Reference direction of Earth's magnetic field
101    hx = mx * q0q0 - _2q0my * q3 + _2q0mz * q2 + mx * q1q1 + _2q1 * my *
        q2 + _2q1 * mz * q3 - mx * q2q2 - mx * q3q3;
102    hy = _2q0mx * q3 + my * q0q0 - _2q0mz * q1 + _2q1mx * q2 - my * q1q1
        + my * q2q2 + _2q2 * mz * q3 - my * q3q3;
103    _2bx = sqrt(hx * hx + hy * hy);
104    _2bz = -_2q0mx * q2 + _2q0my * q1 + mz * q0q0 + _2q1mx * q3 - mz *
        q1q1 + _2q2 * my * q3 - mz * q2q2 + mz * q3q3;
105    _4bx = 2.0f * _2bx;
106    _4bz = 2.0f * _2bz;
107
108    // Gradient decent algorithm corrective step
109    s0 = -_2q2 * (2.0f * q1q3 - _2q0q2 - ax) + _2q1 * (2.0f * q0q1 +
        _2q2q3 - ay) - _2bz * q2 * (_2bx * (0.5f - q2q2 - q3q3) + _2bz *
        (q1q3 - q0q2) - mx) + (-_2bx * q3 + _2bz * q1) * (_2bx * (q1q2 -
        q0q3) + _2bz * (q0q1 + q2q3) - my) + _2bx * q2 * (_2bx * (q0q2 +
        q1q3) + _2bz * (0.5f - q1q1 - q2q2) - mz);
110    s1 = _2q3 * (2.0f * q1q3 - _2q0q2 - ax) + _2q0 * (2.0f * q0q1 +
        _2q2q3 - ay) - 4.0f * q1 * (1 - 2.0f * q1q1 - 2.0f * q2q2 - az) +
        _2bz * q3 * (_2bx * (0.5f - q2q2 - q3q3) + _2bz * (q1q3 - q0q2)
        - mx) + (_2bx * q2 + _2bz * q0) * (_2bx * (q1q2 - q0q3) + _2bz *
        (q0q1 + q2q3) - my) + (_2bx * q3 - _4bz * q1) * (_2bx * (q0q2 +
        q1q3) + _2bz * (0.5f - q1q1 - q2q2) - mz);
111    s2 = -_2q0 * (2.0f * q1q3 - _2q0q2 - ax) + _2q3 * (2.0f * q0q1 +
        _2q2q3 - ay) - 4.0f * q2 * (1 - 2.0f * q1q1 - 2.0f * q2q2 - az) +
        (-_4bx * q2 - _2bz * q0) * (_2bx * (0.5f - q2q2 - q3q3) + _2bz *
        (q1q3 - q0q2) - mx) + (_2bx * q1 + _2bz * q3) * (_2bx * (q1q2 -
        q0q3) + _2bz * (q0q1 + q2q3) - my) + (_2bx * q0 - _4bz * q2) * (
        _2bx * (q0q2 + q1q3) + _2bz * (0.5f - q1q1 - q2q2) - mz);
112    s3 = _2q1 * (2.0f * q1q3 - _2q0q2 - ax) + _2q2 * (2.0f * q0q1 +
        _2q2q3 - ay) + (-_4bx * q3 + _2bz * q1) * (_2bx * (0.5f - q2q2 -
        q3q3) + _2bz * (q1q3 - q0q2) - mx) + (-_2bx * q0 + _2bz * q2) * (
        _2bx * (q1q2 - q0q3) + _2bz * (q0q1 + q2q3) - my) + _2bx * q1 * (
        _2bx * (q0q2 + q1q3) + _2bz * (0.5f - q1q1 - q2q2) - mz);
113    recipNorm = invSqrt(s0 * s0 + s1 * s1 + s2 * s2 + s3 * s3); //
        normalise step magnitude
114    s0 *= recipNorm;

```

```

115         s1 *= recipNorm;
116         s2 *= recipNorm;
117         s3 *= recipNorm;
118
119         // Apply feedback step
120         qDot1 -= beta * s0;
121         qDot2 -= beta * s1;
122         qDot3 -= beta * s2;
123         qDot4 -= beta * s3;
124     }
125
126     // Integrate rate of change of quaternion to yield quaternion
127     q0 += qDot1 * (1.0f / sampleFreq);
128     q1 += qDot2 * (1.0f / sampleFreq);
129     q2 += qDot3 * (1.0f / sampleFreq);
130     q3 += qDot4 * (1.0f / sampleFreq);
131
132     // Normalise quaternion
133     recipNorm = invSqrt(q0 * q0 + q1 * q1 + q2 * q2 + q3 * q3);
134     q0 *= recipNorm;
135     q1 *= recipNorm;
136     q2 *= recipNorm;
137     q3 *= recipNorm;
138 }
139
140 //
-----
141 // IMU algorithm update
142
143 void MadgwickAHRSupdateIMU(float gx, float gy, float gz, float ax, float ay,
144     float az) {
145     float recipNorm;
146     float s0, s1, s2, s3;
147     float qDot1, qDot2, qDot3, qDot4;
148     float _2q0, _2q1, _2q2, _2q3, _4q0, _4q1, _4q2, _8q1, _8q2, q0q0, q1q1,
149         q2q2, q3q3;
150
151     // Rate of change of quaternion from gyroscope
152     qDot1 = 0.5f * (-q1 * gx - q2 * gy - q3 * gz);
153     qDot2 = 0.5f * (q0 * gx + q2 * gz - q3 * gy);
154     qDot3 = 0.5f * (q0 * gy - q1 * gz + q3 * gx);
155     qDot4 = 0.5f * (q0 * gz + q1 * gy - q2 * gx);
156
157     // Compute feedback only if accelerometer measurement valid (avoids NaN
158     // in accelerometer normalisation)
159     if(!((ax == 0.0f) && (ay == 0.0f) && (az == 0.0f))) {
160
161         // Normalise accelerometer measurement
162         recipNorm = invSqrt(ax * ax + ay * ay + az * az);
163         ax *= recipNorm;
164         ay *= recipNorm;
165         az *= recipNorm;
166
167         // Auxiliary variables to avoid repeated arithmetic

```

```

165     _2q0 = 2.0f * q0;
166     _2q1 = 2.0f * q1;
167     _2q2 = 2.0f * q2;
168     _2q3 = 2.0f * q3;
169     _4q0 = 4.0f * q0;
170     _4q1 = 4.0f * q1;
171     _4q2 = 4.0f * q2;
172     _8q1 = 8.0f * q1;
173     _8q2 = 8.0f * q2;
174     q0q0 = q0 * q0;
175     q1q1 = q1 * q1;
176     q2q2 = q2 * q2;
177     q3q3 = q3 * q3;
178
179     // Gradient decent algorithm corrective step
180     s0 = _4q0 * q2q2 + _2q2 * ax + _4q0 * q1q1 - _2q1 * ay;
181     s1 = _4q1 * q3q3 - _2q3 * ax + 4.0f * q0q0 * q1 - _2q0 * ay - _4q1 +
        _8q1 * q1q1 + _8q1 * q2q2 + _4q1 * az;
182     s2 = 4.0f * q0q0 * q2 + _2q0 * ax + _4q2 * q3q3 - _2q3 * ay - _4q2 +
        _8q2 * q1q1 + _8q2 * q2q2 + _4q2 * az;
183     s3 = 4.0f * q1q1 * q3 - _2q1 * ax + 4.0f * q2q2 * q3 - _2q2 * ay;
184     recipNorm = invSqrt(s0 * s0 + s1 * s1 + s2 * s2 + s3 * s3); //
        normalise step magnitude
185     s0 *= recipNorm;
186     s1 *= recipNorm;
187     s2 *= recipNorm;
188     s3 *= recipNorm;
189
190     // Apply feedback step
191     qDot1 -= beta * s0;
192     qDot2 -= beta * s1;
193     qDot3 -= beta * s2;
194     qDot4 -= beta * s3;
195 }
196
197 // Integrate rate of change of quaternion to yield quaternion
198 q0 += qDot1 * (1.0f / sampleFreq);
199 q1 += qDot2 * (1.0f / sampleFreq);
200 q2 += qDot3 * (1.0f / sampleFreq);
201 q3 += qDot4 * (1.0f / sampleFreq);
202
203 // Normalise quaternion
204 recipNorm = invSqrt(q0 * q0 + q1 * q1 + q2 * q2 + q3 * q3);
205 q0 *= recipNorm;
206 q1 *= recipNorm;
207 q2 *= recipNorm;
208 q3 *= recipNorm;
209 }
210
211 //
    -----
212 // Fast inverse square-root
213 // See: http://en.wikipedia.org/wiki/Fast\_inverse\_square\_root
214

```

```
215 float invSqrt(float x) {
216     float halfx = 0.5f * x;
217     float y = x;
218     long i = *(long*)&y;
219     i = 0x5f3759df - (i>>1);
220     y = *(float*)&i;
221     y = y * (1.5f - (halfx * y * y));
222     return y;
223 }
224
225 //
```

=====

```
226 // END OF CODE
227 //
```

=====

C. Planos

